

CEREBRO

Threat & Misinformation Intelligence Platform

Technical Report — architecture, build record & integration brief

Prepared as a complete record of the CEREBRO system

Role: the sensory / feature-extraction & data layer for the Apollo predictive system

Status: built, deployed, and collecting data (live)

Stack: React 19 + Vite · FastAPI · PostgreSQL 16 + pgvector · Python ML

Hosting: Vercel (frontend) · Render (backend + database)

Includes the live URLs and a shared demo login for the team. Application signing/encryption keys, the Fact Check API key and the database password are omitted (held in Render's secret manager).

Executive Summary

CEREBRO is an evidence-producing threat-and-misinformation intelligence platform. A user submits an email, a news claim, or network traffic, and CEREBRO returns a verdict together with the evidence behind it: the exact features it measured, the named model or heuristic that produced the score, and — for misinformation — links to real published fact-checks the reader can open and verify. Its governing principle is that **deterministic code and trained models make the decision, while a language model is used only to explain** — which makes every verdict measurable, reproducible, auditable, and able to run without shipping private data to a third party.

The platform is fully built and deployed. The frontend runs on Vercel, the API on Render, and a PostgreSQL 16 database with the pgvector extension stores every analysis. CEREBRO is intentionally scoped as the **sensory and feature-extraction layer** for a larger predictive system (the *Apollo* project): it turns raw text and telemetry into the engineered features that a forecasting and classification core will consume. This report is the complete record of what CEREBRO is, what was built, the technology used, where everything lives, what it draws on, and how it connects to Apollo.

■ TRY IT / TEAM ACCESS

App: <https://cerebro-sandy-beta.vercel.app> **Login:** `demo@cerebro.app / cerebro-demo-2026`

API + docs: <https://cerebro-api-nmah.onrender.com/docs> **Code:** <https://github.com/kashaffatimajaffrey-design/CEREBRO>

1. Purpose & the Gap It Fills

The first iteration of the system asked a general-purpose chatbot to assign a “credibility score.” For a security tool that approach fails in four decisive ways, each of which CEREBRO was rebuilt to fix:

- **Invented evidence.** The chatbot cited sources that did not exist. CEREBRO only cites documents and fact-checks that actually resolve.
- **Unmeasurable.** A chatbot score has no precision, recall, or confusion matrix. CEREBRO's scores come from feature extractors and models that can be measured and defended.
- **Not reproducible.** The same input produced different answers. CEREBRO's deterministic pipeline yields the same result every time.
- **Privacy.** Inbox contents were sent to a third party. CEREBRO's detection runs locally; no message needs to leave the operator's infrastructure.

The gap CEREBRO fills is therefore the difference between an *opinion* and *auditable evidence*. It provides, for every verdict, the features that drove it, the model or heuristic version that computed it, the supporting sources, and an explicit statement of confidence — including the ability to say “insufficient evidence” rather than guessing.

2. Governing Principle

A single rule shapes the entire architecture:

“Deterministic code and trained models decide. The language model only explains.”

The language model is optional and confined to two linguistic jobs: segmenting text into checkable claims, and writing an analyst-readable summary of findings that already exist. It never scores, classifies, or decides. Consequently, with the language model switched off entirely, every score is identical — the detection does not depend on it.

3. System Architecture

CEREBRO is deployed as two independent services that communicate over HTTPS and a WebSocket, backed by a managed PostgreSQL database. The layered topology is:

Layer	Runs on	Responsibility
Presentation	Vercel (static SPA)	React dashboard, the four scanners, authentication UI, live threat-map background
Application / API	Render (Docker)	FastAPI gateway: auth, feature extraction, scoring, persistence, realtime fan-out
Intelligence (ML)	in the API process	Email forensics, RAG verification, anomaly detection, forecasting scaffold, LLM providers
Data	Render PostgreSQL 16 + pgvector	Users, tenants, every analysis, detections stream, model registry, forecasts

Requests flow front-to-back: the SPA calls the API; the API extracts deterministic features, scores them, persists the result, and emits a database notification that is pushed back to the browser over a WebSocket so the dashboard updates without polling. Multi-tenancy is enforced in the database itself through row-level security, not in application code — if the tenant context is ever missing, the database returns zero rows, which is the safe direction to fail.

4. Technology Stack (complete)

Area	Technology	Why
Frontend	React 19, TypeScript, Vite, Tailwind-style CSS, Recharts, Framer Motion, Canvas	Fast SPA, type safety, live charts and the animated threat-map background
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2	Async API that shares the ML runtime; automatic OpenAPI docs
Database access	asyncpg (direct), PostgreSQL 16, pgvector	Native \$1,\$2 queries, LISTEN/NOTIFY realtime, per-connection RLS scoping, vector search
Auth / crypto	stdlib scrypt, HMAC-SHA256 JWT, AES-256-GCM (cryptography)	Memory-hard hashing, dependency-light tokens that reject alg=none, encrypted tokens at rest
ML — classification	scikit-learn, LightGBM, (RoBERTa/transformers optional)	Calibrated phishing classifier; RoBERTa heads planned in Apollo
ML — retrieval / NLI	BM25 + vector fusion, cross-encoder rerank, NLI stance	Hybrid evidence retrieval with citations that resolve

Area	Technology	Why
ML — anomaly	IsolationForest, numpy autoencoder, DBSCAN	Unsupervised network detection + incident clustering
ML — forecasting	pytorch-forecasting (TFT), torch	Quantile threat-volume forecasting (scaffold; matures in Apollo)
External data	Google Fact Check Tools API	Real, citable misinformation verdicts with source links
Infra / deploy	Docker, Render (blueprint), Vercel, GitHub Actions CI	Reproducible builds; one-click backend + database; CI against a real Postgres

5. Functional Modules

5.1 Email forensics

Parses a full RFC 5322 message from raw bytes and extracts roughly 35 deterministic signals: SPF, DKIM and DMARC results and DKIM-domain alignment against the From address; Reply-To, Return-Path and display-name spoofing; lookalike / typosquatted sender domains; Unicode homograph detection; anchor-text mismatches, IP-literal URLs, shorteners and credential-in-URL tricks; attachment risk; and manipulation-language density. Each finding carries a weight and a plain-English reason. A transparent heuristic prior produces the score today; a calibrated classifier trained on the SpamAssassin corpus (held-out F1 $\approx$ 0.97) can be loaded to replace it, at which point the response is labelled `model` instead of `heuristic`.

5.2 Misinformation verification

For a submitted claim, CEREBRO first queries the Google Fact Check Tools API and, when real published fact-checks exist, derives the credibility from what fact-checkers actually rated it and returns their article URLs as clickable sources. When no fact-check exists, it falls back to a hybrid retrieval + NLI check against an evidence corpus, and finally to a transparent linguistic-credibility heuristic (sensationalism, conspiracy markers, sourcing/attribution signals). Every result states which path produced it, and unverifiable claims are reported as such rather than guessed.

5.3 Network anomaly detection

Scores network flows for intrusions. A transparent rule-based triage flags SYN floods, oversized packets, ICMP floods and scanning sources on the flows it is given; when a trained unsupervised model (IsolationForest + autoencoder, fitted on benign traffic from CIC-IDS2017) is registered, it takes over and clusters related alerts into incidents. Detection is computed from the traffic, never planted.

5.4 Forecasting (scaffold)

A Temporal Fusion Transformer pipeline forecasts threat volume ahead with quantile intervals (p10/p50/p90). It is intentionally gated: it refuses to fit until enough real history has accumulated, and the API reports “insufficient history” rather than fabricating a curve. This module is the primary hand-off point to the Apollo predictive core.

5.5 Dashboard, auth & realtime

The dashboard renders live counts and charts computed from the detections table, updated over a WebSocket driven by database notifications. Authentication is multi-tenant with bcrypt-hashed passwords, signed JWT sessions delivered as bearer tokens (so they work on mobile browsers that block cross-site

cookies), and encrypted OAuth token storage.

6. Data & Persistence — what is collected and where it lives

Every analysis is written to PostgreSQL. This is deliberate: the stored data is the dataset the parent Apollo project will train on. The database lives on Render (managed PostgreSQL 16 with pgvector) and the schema is applied automatically on first boot. The tables that matter most:

Table	What it stores	Primary use
detections	Unified event stream: timestamp, tenant, module, threat type, risk score, summary	Time-series input for forecasting; drives the dashboard metrics
email_analyses	Full forensic record: auth results, from-domain, features, indicators, verdict	Labelled training data for the phishing classifier
verdicts	Misinformation verdicts with confidence, calibration and evidence	Misinformation labels + calibration
anomalies / incidents	Network anomaly scores, feature attribution, clustered incidents	Network model outputs
forecasts	Quantile forecasts (p10/p50/p90) per horizon	Written by the forecasting layer; rendered on the dashboard
model_registry	Model name, version, task, metrics, active flag	Shared model versioning across CEREBRO and Apollo
analyst_feedback	Analyst corrections to model outputs	The correction-to-retrain flywheel
tenants / users / sessions	Multi-tenant accounts and login sessions	Authentication and row-level isolation

Row-level security keys every tenant-scoped table to the authenticated tenant, so one operator can never read another's data. Retrieving the collected data for downstream modelling is a plain SQL query against the detections and email_analyses tables.

7. External Integrations & Data Sources

CEREBRO draws on the following, all free or open:

Source	What it provides	Where it is used
Google Fact Check Tools API	Real fact-checks (Full Fact, FactCheck.org, PolitiFact, AFP...) with ratings + URLs	Misinformation verification (live)
Apache SpamAssassin corpus	~7,400 real spam/ham emails	Training the phishing classifier
CIC-IDS2017 / UNSW-NB15	Labelled network flows for benign/attack traffic	Training & evaluating the anomaly detector
NAB (NYC-taxi) series	Real time-series with seasonality	Backtesting the forecasting pipeline
FEVER / LIAR	Labelled claims for stance / classification	Planned RoBERTa fine-tuning (Apollo)

What it is *supposed to pull from* at scale, once ingestion workers are added, is live telemetry: RSS / news feeds and IMAP mailboxes for text, and Zeek / Suricata for network flows. Today those enter through the API rather than an automated worker — a known, documented gap.

8. Security Model

- **Passwords** hashed with scrypt (memory-hard); never stored or logged in plaintext; constant-time verification.
- **Sessions** are HMAC-SHA256 JWTs that explicitly reject the a lg=none forgery, verify signature, expiry and required claims.
- **OAuth tokens** are AES-256-GCM encrypted at rest and bound to their owner, so a database dump cannot yield live mailbox access.
- **Tenant isolation** is enforced by PostgreSQL row-level security, not application filters.
- **Error handling** never leaks internals to the client; a strict Content-Security-Policy is applied to the frontend.
- **Secrets** (signing key, encryption key, API keys, database URL) live only in the host's secret manager and are deliberately excluded from this document.

9. Where Everything Lives

The system is public-facing and running. The live locations (no credentials reproduced here):

Component	Location / URL
Live application (frontend)	https://cerebro-sandy-beta.vercel.app
Backend API + interactive docs	https://cerebro-api-nmah.onrender.com (/docs, /health, /ready)
Demo login (reviewers & team)	demo@cerebro.app / cerebro-demo-2026
Source code (single source of truth)	https://github.com/kashaffatimajaffrey-design/CEREBRO
Database	Render managed PostgreSQL 16 + pgvector (service: cerebro-db)
Hosting	Render (service: cerebro-api) + Vercel (project: cerebro)
CI	GitHub Actions — lint, type-check, schema apply, test against Postgres

The demo login above is a shared demo account for reviewers/teammates. The application signing key, token-encryption key, Fact Check API key and database password are NOT printed here — they live in the Render secret manager.

10. Integration with Apollo (the predictive core)

CEREBRO is one half of a two-part system. It is the **sensory and feature layer** — it sees, extracts and records. **Apollo** is the **predictive brain** — a Temporal Fusion Transformer plus RoBERTa and unsupervised models that learn from CEREBRO's data and forecast and classify. The boundary is clean: CEREBRO never re-implements prediction, and Apollo never re-implements feature extraction.

Apollo connects through any combination of four mechanisms, all supported by the existing schema:

- **Shared database (primary).** Apollo reads detections (the hourly time-series its forecaster trains on) and email_analyses (its classifier training data), and writes results back into forecasts and model_registry — which CEREBRO's dashboard already renders.
- **Realtime stream.** A database notification fires on every detection; Apollo can subscribe for online features with no polling.
- **REST API.** The analyze, metrics, flows, stream and forecast endpoints allow loose coupling.
- **Shared model registry.** One table versions models the same way for both projects.

The recommended data topology is a single PostgreSQL instance with separate schemas — CEREBRO's tables and an Apollo schema for training runs, feature snapshots and artifact metadata — plus object storage for model weights referenced from the registry. This avoids any extract-transform step: Apollo's forecasts light up CEREBRO's live dashboard immediately.

11. Current Status — complete vs. gaps

Capability	Status
Frontend, API, database, auth (desktop + mobile), deployment	Complete & live
Email forensics, misinformation (with real fact-checks), network triage	Working in production
Persistence / data collection into PostgreSQL	Complete
Trained classifiers (email / anomaly)	Built as scripts; not loaded in the live instance
Automated ingestion workers (RSS / IMAP / Zeek)	Not built — data enters via the API
TFT forecasting in production	Scaffolded & validated; needs accumulated history

In short: CEREBRO is a working, deployed, data-collecting platform — complete enough to serve as Apollo's data and feature layer — with its remaining work stated honestly rather than hidden.

12. Testing & Verification

The backend carries 67 automated tests, including 23 adversarial security tests (JWT forgery, payload tampering, cross-tenant access, authenticated-encryption tampering — all rejected). The email classifier was trained and measured on a held-out split of real data (precision 0.96, recall 0.97, F1 0.97, AUC-ROC 0.996), and it demonstrably corrects the heuristic. The forecasting pipeline was backtested on a real seven-month series. The frontend type-checks cleanly, and continuous integration runs the whole suite against a real PostgreSQL + pgvector instance on every change.

13. Engineering Record — notable decisions

- **asyncpg over an ORM** — the analytics queries use native placeholders and the realtime path needs LISTEN/NOTIFY, both first-class in asyncpg.
- **Deterministic-first design** — heuristics and models score; the LLM only explains, keeping results measurable and private.
- **Honest fallbacks** — each module degrades transparently (fact-check → retrieval → heuristic; trained model → heuristic prior) and always labels which path ran.

- **Mobile auth** — bearer tokens were adopted because iOS Safari blocks the cross-site cookie the desktop relied on.
- **Resilient deployment** — the platform was brought up on free, no-vendor-lock tiers; where an image could not be pulled on a restricted network, the pgvector image is built from source locally.

End of report. CEREBRO is the eyes and ears; Apollo is the brain. This document records the former in full so the latter can be built on a clear foundation.